

CLAMP: Contrastive Language Model Prompt-tuning

Piotr Teterwak¹ Ximeng Sun¹ Bryan A. Plummer¹ Kate Saenko¹ Ser-Nam Lim²
Boston University¹ University of Central Florida²
{piotrt, sunxm, bplum, saenko}@bu.edu sernam@ucf.edu

Abstract

Large language models (LLMs) have emerged as powerful general-purpose interfaces for many machine learning problems. Recent work has adapted LLMs to generative visual tasks like image captioning, visual question answering, and visual chat, using a relatively small amount of instruction-tuning data. In this paper, we explore whether modern LLMs can also be adapted to classifying an image into a set of categories. First, we evaluate multimodal LLMs that are tuned for generative tasks on zero-shot image classification and find that their performance is far below that of specialized models like CLIP. We then propose an approach for light fine-tuning of LLMs using the same contrastive image-caption matching objective as CLIP. Our results show that LLMs can, indeed, achieve good image classification performance when adapted this way. Our approach beats state-of-the-art mLLMs by 13% and slightly outperforms contrastive learning with a custom text model, while also retaining the LLM’s generative abilities. LLM initialization appears to particularly help classification in domains under-represented in the visual pre-training data.

1. Introduction

In the past several years the capabilities of Large Language Models (LLMs) have progressed from interesting to extremely useful. This success is equal parts data and compute; LLaMA2 [48] is trained on 2 trillion tokens and millions of GPU-hours. Recent work has adapted LLMs to multimodal inputs such as images, by learning a mapping from a frozen visual backbone to a continuous token used as input to the LLM. Such multimodal LLMs (mLLMs) include LLaVA [29], MiniGPT [59], Instruct-BLIP [9], and GPT-4V [35] and have impressive image captioning and question-answering capabilities. For example, several of these models can effectively answer why a picture of someone ironing clothing on the back of a moving taxi-cab is unusual (see GPT-4 Technical Report[35], Table 16.)

Yet generative tasks like visual-question answering are not the only place where LLM world-knowledge could

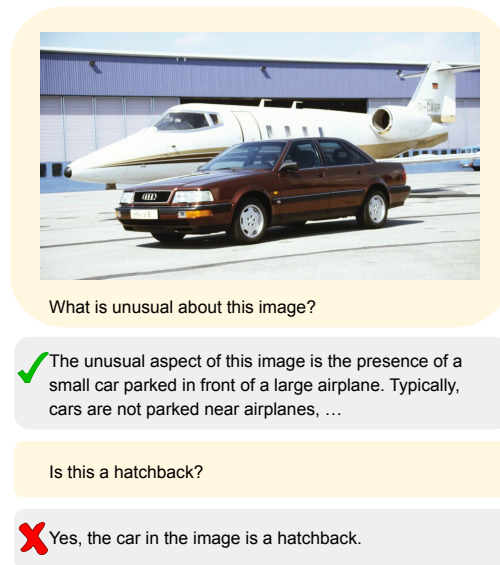


Figure 1. State-of-the-art multimodal LLMs excel at generative visual tasks like answering questions that involve common sense, but underperform on standard image classification tasks like predicting the car type (generated using LLaVA1.5 [29]). On a suite of 24 zero-shot image classification datasets they underperform SOTA zero-shot classification models like CLIP [38] by 13%. In this paper, we present an approach to improve classification while retaining the LLM’s generative abilities.

be useful. In this paper we ask, **can multimodal LLMs also perform zero-shot classification tasks?** For instance, given a list of vehicle types and an image, can an mLLM identify which type of vehicle is present (Fig. 1)? We find that the answer is, surprisingly, not well. For example, state-of-the-art mLLMs get under 10% top-1 classification accuracy on Stanford Cars [22], while CLIP achieves over 80% (Tab. 1). This is counter-intuitive; LLMs have been proven to encode extensive knowledge about the world. In fact, one can even ask an LLM what an object looks like, and get a reasonable answer.

We hypothesize that this weakness is due not to a lack of capability of the LLM but to improper training, and that the generative captioning and instruction-tuning objectives do

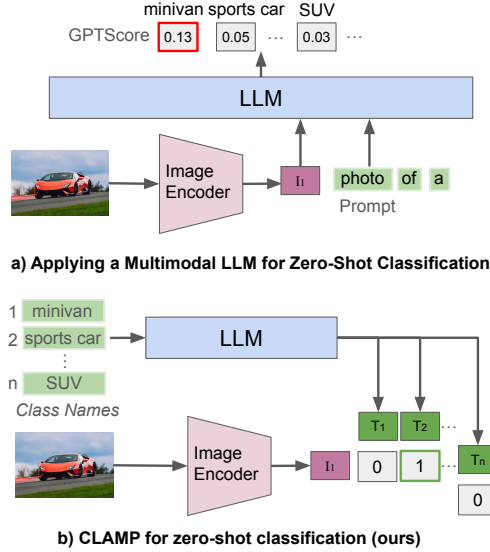


Figure 2. **Adapting LLMs for image classification:** a) Applying prior multimodal LLMs such as LLaVA [29] and MiniGPT [59] to classification by computing the GPTScore [27] has poor accuracy; b) Our approach CLAMP achieves high accuracy by lightly fine-tuning the LLM with a contrastive image-caption objective.

not enable effective classification. To build some intuition for this, consider that LLMs contain a powerful language prior which may improve visual-question answering even without image features; for example, the correct answer may simply have higher-likelihood in the language model, as illustrated by Visual GPT-Score [27]. In that work, it was shown that SoTA performance can be achieved on several vision-language benchmarks by ignoring the context image completely and just computing the likelihood of the text sequences using the language model.

Given our hypothesis that generative training is not sufficient for discriminative image tasks, but that LLMs still encode knowledge useful for those tasks, we propose to use an LLM as a text encoder together with contrastive language-image pre-training (e.g. CLIP [38]). We call our method *Contrastive Language Model Prompt-tuning (CLAMP)* (Fig. 2.) We find that by updating a minority of the LLM parameters, our method can approach the performance of zero-shot CLIP trained on much larger scale data while outperforming a text-encoder trained from scratch on the same data. This finding highlights the benefits of using Large Language Models; LiT [55] finds that initializing to pre-trained (but smaller scale) LMs is not helpful for zero-shot classification in English. Surprisingly, we’re also able to mostly retain the generative capabilities of the language model, pointing towards universal generative and discriminative models.

In summary, our contributions are as follows:

- We show that SOTA multimodal LLMs are neither accu-

rate nor efficient at zero-shot image classification.

- We create a new form of multimodal LLM which replaces the text encoder of a contrastive vision-language model with a pretrained LLM and updates a limited set of parameters using a contrastive loss.
- We show that the LLM initialization allows for improved performance in domains under-represented in the pre-training data, and that our CLAMP retains generative abilities despite our fine-tuning.

2. Related Work

Contrastive Training. Contrastive training has become a popular training technique. It aims to align positive pairs in embedding space, while pushing negative pairs apart. Therefore, this type of method strongly depends on how positive and negative pairs are formed. Some methods contrast images with their augmented selves [3, 21, 34, 47], while Vision-Language models (e.g. CLIP [38]) use a text-encoder and contrast against text captions [38, 55, 56], therefore learning a joint image-text embedding. This joint embedding space enables Vision-Language models to classification in the zero-shot setting, without training data with explicit labels. In this work, we add a contrastive objective to fine-tune an LLM, and therefore enable the LLM to function as an effective zero-shot visual classifier.

Large Language Models. Large Language Models are transformers [49], scaled to billions of parameters, trained generatively on trillions of tokens. LLMs are commonly trained with next-token prediction. The success of LLM-based chatbots such as ChatGPT has inspired a great number of efforts to create both general [6, 35, 48, 58] and domain specific [41, 50] language models. Instruction-tuning additionally fine-tunes LLMs with (instruction, output) pairs, which makes LLMs more useful for specific tasks. In this work, we explore adapting an LLM to image classification.

Regularized Fine-tuning. Also known as Parameter-Efficient Fine Tuning (PEFT), regularized fine-tuning aims to update only a subset of the parameters in a pre-trained model. This allows one to keep around only a small number of parameters for each specialized task a model is fine-tuned for, to use less memory in the training process, and to not unlearn knowledge present in the pre-trained model. Common approaches include low-rank updates of existing matrices (LoRA [17]), prompt and prefix tuning [25, 26, 30], and inserting additional trainable layers into the middle of the network (Adapters [16]). In our work, we use prompt tuning and LoRA to adapt the LLM to the contrastive text-image task.

Multimodal Large Language Models. Until recently, large LMs have operated exclusively in the text domain. However, a plethora of models such as LLaVA [29], GPT-4V [35], MiniGPT [59], Flamingo [1], and BLIP [9] now

can take images as input. These models typically use a pre-trained vision encoder and a small set of learnable parameters to map from vision encoder output to LLM input. They are typically trained with a captioning loss, and can additionally be tuned with an instruction-tuning objective. The base LLM can be frozen [59] or updated [48]. Multimodal LLMs work well for tasks such as visual question answering or captioning. However, we show that they are generally ineffective for zero-shot classification.

3. Contrastive LLanguage Model Prompting

We propose *Contrastive LLanguage Model Prompt-tuning (CLAMP)*, a method to adapt Large Language Models to classification via prompting and contrastive learning. Our approach is inspired by state-of-the-art vision-language models trained with the contrastive objective, i.e. CLIP. The objective encourages a strong alignment between text and image representations in a joint space, enabling effective-zero shot classification. We propose to align LLM representations with visual representations using a contrastive loss. The question is, how to prompt the LLM to output a representation that is useful for image classification? After all, the information needed to predict the next token in a caption (language modeling) is not necessarily the same as that to encode the visual category in that same caption.

CLAMP accomplishes this in three main ways. First, it learns a pooling over the LLM’s output representations. We implement this with attention-pooling (Section 3.1.2), with a query token that learns to aggregate the output representations of an encoded caption. However, output pooling is insufficient to get the LLM to produce useful information that is embedded in the remaining network layers. To overcome this, CLAMP adds learned read-only prompts [25] (Section 3.1.1), which serve as special-purpose discriminative tokens in a model pre-trained for generation. Finally, we find that slight modification of the parameters improves zero-shot performance further, accomplished with LoRA, which learns low-rank delta updates of certain layers (Section 3.1.3.)

Section 3.2 details the text prompt used in training and Section 3.3 describes the training objective, which includes distillation of a stronger CLIP model into our trained model, inspired by Dime-FM [44].

3.1. Architecture

In our experiments, we build on top of Vicuna-1.5 [5], which is a LLaMA-2 based instruction-tuned model. We align the Vicuna with a pre-trained visual encoder contrastively. In order to do so without risking overfitting, we only add a small number of trainable modules including Output Attention Pooling (Section 3.1.2) at the output, Read-Only Prompts (Section 3.1.1) at the input, and LoRA

(Section 3.1.3) in the intermediate layers. Similar to many mLLMs, we freeze the visual encoder.

3.1.1 Read-only Prompts

Frozen LLMs are generative models, therefore in order to add discriminative capabilities we need to modify the model. However, doing so without harming the good representations learned during pre-training requires care, and one method used to do so is prompt tuning. We do so by appending learnable prompts to the end of the text sequence, which function as special-purpose tokens for discriminative tasks.

$$x = [x_{\text{text}}; \{p_{\text{learnable}}\}] \quad (1)$$

where x is the input sequence to the LLM, x_{text} is the tokenized language and $p_{\text{learnable}}$ are the learnable prompts. The LLM we use uses causal attention, meaning each position’s representation can only attend to tokens previously seen in the sequence. When attention is bi-directional, the intermediate representations of x_{text} can shift with changes in $p_{\text{learnable}}$. With causal attention, however, $p_{\text{learnable}}$ can change without updating intermediate representations of x_{text} . Such a technique is known as read-only prompting [25], and it is shown to improve generalization of prompt-tuning of contrastive vision-language models to unseen classes, due to minimizing overfitting. To produce the text embedding, we perform output attention pooling over the output representations of the learnable prompt tokens.

3.1.2 Output Attention Pooling

Transformer networks take as input variable-length sequences, and output sequences of that same length. However, the joint image-text embedding space of vision-language models requires a representation of a fixed length. Therefore, some kind of pooling of tokens is needed. CLIP [38] accomplishes this by simply using the last token in the sequence as the representation of the text. We instead use the read-only prompts described in the previous section. However, since we use many of those prompts, we still need to pool them to a fixed length. We choose to leverage a form of pooling that uses the multi-headed attention mechanism:

$$\text{Attention}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W_O \quad (2)$$

where

$$\text{head}_i = \text{Attention}(QW_{Q_i}, KW_{K_i}, VW_{V_i}) \quad (3)$$

and

$$\text{Attention}(Q, K, V) = \text{Softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \quad (4)$$

In our attention pooling mechanism, we learn the query Q and the K and V are simply output tokens of the LLM. In this way, we can learn a per-sample linear combination of LLM outputs.

3.1.3 LoRA

While Attention Pooling and Read-Only Pooling are important architectural components in extracting discriminative features from text sequences, discriminative performance can further be improved if the network parameters can be lightly updated. One option is full-finetuning of the network parameters. This, however, has two large drawbacks. First, it has heavy memory requirements. In particular, for the large-batch training necessary for effective contrastive learning, full fine-tuning requires the storage of intermediate activation. Secondly, we go back to the motivation of not catastrophically forgetting information present in the LLM in our contrastive training. Therefore, we leverage LoRA, which learns low rank updates to pretrained models:

$$W = W_0 + BA^T \quad (5)$$

where W are the updated weights in $\mathbb{R}^{d \times d}$, and B and A are in $\mathbb{R}^{d \times k}$.

As is common, we only update multi-headed attention layers with our LoRA updates. We use a rank of 16, an α of 16, a dropout value of 0.1, and no bias terms.

3.2. Wrapper prompts

Vicuna is trained with a very specific prompt template:

“A chat between a curious user and an artificial intelligence assistant. The assistant gives helpful, detailed, and polite answers to the user’s questions.

USER: Hello!

ASSISTANT: Hello!”.

In order to follow this format of USER followed by ASSISTANT, we use the following prompt template for our training:

“USER: A photo of a {}. A {} has the following visual attributes.

ASSISTANT: 1.”

Where the $\{\}$ is replaced with the caption of a particular sample. Auto-regressive decoding of this prompt would result in a description of what the image with the caption may look like. Although auto-regressive decoding in the training loop is prohibitively expensive; Xiao et al. [51] show that much information about the decoded sequence is present in the initial tokens, and therefore prompting the LLM to describe the caption is a reasonable way to surface visual information.

3.3. Contrastive Training of LLMs

So far, we have discussed the architectural components of our method. We now turn our attention to the training methodology. Following CLIP [38], we use image-text and text-image contrastive losses:

$$\mathcal{L}_{img2txt} = - \sum_i \log \frac{\exp(\tau v_i^T t_i)}{\sum_j \exp(\tau v_i^T t_j)} \quad (6)$$

$$\mathcal{L}_{txt2img} = - \sum_i \log \frac{\exp(\tau t_i^T v_i)}{\sum_j \exp(\tau t_i^T v_j)} \quad (7)$$

$$\mathcal{L}_{contrastive} = \mathcal{L}_{img2txt} + \mathcal{L}_{txt2img} \quad (8)$$

where t is a text representation, v is a visual representation, and i and j index the batch. τ is a learned temperature parameter.

In addition, we add a distillation loss term to the training, similar to Dime-FM [44]. Following [44], we first compute teacher similarity matrix S_t and student similarity matrix S_s . These are computed using the vision and text encoders over samples in the batch, as in the contrastive loss. Defining row-wise indexing variable as i and column-wise indexing variable as j , the distillation loss is defined as:

$$\mathcal{L}_{dist} = \sum_i KL(\tau_d S_t || \tau_d S_s) + \sum_j KL(\tau_d S_t^T || \tau_d S_j^T) \quad (9)$$

Finally, our loss becomes:

$$\mathcal{L}_{tot} = \mathcal{L}_{contrastive} + \mathcal{L}_{dist} \quad (10)$$

4. Experiments

We evaluate our models on zero-shot classification tasks in order to test the discriminative ability of our adapted large language models. We compare to two sets of models; **a.)** To existing multimodal Large Language Models LLaVA [29] and Mini-GPT [59], to demonstrate how naive usage of existing mLLMs does not meet the expectations of modern zero-shot classifiers, and **b.)** To a contrastive vision-language model where we retrain the parameters of the text encoder, also known as LiT [55]. We select a diverse set of 24 datasets to evaluate zero-shot performance, including fine-grained datasets such as Stanford Cars [22] and Aircraft [31], natural but challenging datasets such as EuroSAT[14] and ImageNet[10], and synthetic data such as Kitti [13].

Since we aim for our models to retain their *generative* abilities in addition to gaining discriminative abilities, we show how our models can continue to generate effectively with both qualitative examples and quantitative metrics, eg. on MMLU[15]. This shows the universality of our model.

	ImageNet 1k	Cars	Country211	FGVC Aircraft	GTSRB	MNIST	RenderedSST2	STL10	Sun397	Caltech101	CIFAR 10	CIFAR 100	CLEVR Distance	CLEVR Count	Diabetic Ret.	DMLAB	DTD	EuroSAT	Flowers	Kitti	PatchCamelyon	Pets	Resisc45	SVHN	Average
LLaVa v1.5	30.7	5.6	3.6	6.7	34.6	84.9	62.8	98.2	43.3	65.1	12.5	54.8	9.1	12.5	2.3	21.3	26.1	33.4	12.5	37.7	49.2	15.1	39.1	54.2	34.0
minigt4	24.0	8.9	4.8	6.5	31.7	58.4	52.9	93.1	36.4	63.1	30.2	60.8	9.1	12.7	2.3	12.4	21.1	31.4	30.2	31.5	50.8	26.8	26.7	45.5	32.1
LiT [55]	64.0	74.6	11.7	9.4	43.2	36.4	48.4	97.6	65.5	82.2	67.9	91.9	11.0	13.3	2.3	19.7	36.1	47.7	66.5	18.7	56.4	80.8	50.4	17.0	46.4
CLAMP(Ours)	63.0	78.7	12.7	16.3	31.8	45.7	51.5	96.4	66.4	81.1	65.3	88.4	11.2	14.9	4.4	14.9	49.3	47.8	64.9	24.2	50.2	78.3	42.6	28.3	47.0

Table 1. **Comparison of CLAMP with multimodal Large Language Models and LiT.** We find that a contrastive finetuning of LLMs outperforms current SOTA multimodal LLMs such as LLaVa v1.5 [29] and minigt4[59] by 13%. LLaVA tends to outperform on datasets needing OCR (such as MNIST, RenderedSST, and SVHN), but this is more than made up for with major performance degradation on fine-grained datasets such as Cars and Aircraft. CLAMP replaces the text encoder with an LLM, and lightly updates it with a contrastive loss. This is much more performant on average than current multimodal LLMs for zero-shot classification, and even outperforms or is on par with a vision-language model trained on the same data (LiT). In Table 2, we show that CLAMP strongly outperforms LiT in regimes where the target data is not well covered in pre-training.

4.1. Multi-modal LLMs as Zero-shot Classifiers

We compare our method to the multimodal Large Language Models by computing the likelihood of an image-class caption sequence, following Flamingo[1] and VisualGPTScore[27]. In more detail, given an image I and a set of class labels, for each class label c consisting of one or more tokens, we compute the likelihood of sequence $P(\text{'A photo of } c' | I)$. We additionally normalize by the sequence length, because longer sequences generally have lower likelihood and this is not a sensible prior for classification. Then, the score for each class is:

$$\frac{1}{N} \sum_{i=1}^N \log(P(t_i | t_{0..i-1}; I)) \quad (11)$$

where N is the sequence length, I is the image to be classified, and t are the sequence tokens. To classify, we simply select the score with the highest normalized log-likelihood.

4.2. Zero-shot classification

Training and Evaluation Setup: For the text encoder, we use a pre-trained Vicuna13B-1.5[5], which uses LLaMa2 as a base model. We enable attention pooling (Section 3.1.2), Read-Only Prompting (Section 3.1.1) with 24 soft prompts, and LoRA (Section 3.1.3) as the text encoder trainable parameters. For the vision encoder, we use an OpenCLIP pretrained ViT-B-16, and keep it frozen. We train with a data mixture of Conceptual Captions 3M[40], Conceptual Captions 12M[2], a 200-million scale subset of LAION-400M[39], and ImageNet-21k[10]. All of these datasets, except for ImageNet21k, are image-caption datasets. In contrast, ImageNet21k is a dataset of approximately 21000 labels. In order to convert this to an image-caption dataset, we wrap each label in one of the 80 prompt templates used in the CLIP paper [38]. We train for 480 million seen examples, with a batch size of 8192. We use the standard

image-text contrastive loss in addition to distilling a ViT-L-14 trained on DataComp XL [?] into the model. Additional training details are in the supplementary. When we do zero-shot evaluation, we use a prompt template matching the training prompt (Section 3.2).

Baselines:

MiniGPT4 [59]: MiniGPT4 learns a linear mapping between the output of a pre-trained CLIP encoder and a frozen LLM, trained using Conceptual Captions. In our experiments, we use a MiniGPT4 with a 16-bit Vicuna13B model.

LLava-1.5 [28]: LLaVa-1.5 is similar to MiniGPT in architecture, but with a different data mixture. In particular, LLava-1.5 combines academic VQA datasets with an expansion of COCO by ChatGPT [29] to create a vision-language instruction-following dataset. LLaVA-1.5 also finetunes the LLM with LoRA.

LiT[55]: To compare to state-of-the-art contrastive vision-language models, we train a LiT model, which keeps the vision encoder frozen while updating all text encoder parameters. We initialize with a ViT-B-16 CLIP text encoder from scratch, and train on the same data and for the same length as our model. For LiT models, we use an evaluation prompt template of 'A photo of {class}'. We use a single prompt template for evaluating both LiT and CLAMP.

Results: We present the performance of our model in Table 1. We find that on average, we outperform generatively-trained LLaVA-v1.5 and miniGPT-4 by 13% and the LiT baseline by half a percent. This demonstrates that adding a contrastive loss to an LLM, together with light fine-tuning, adds a visual discriminative ability to LLMs which current SOTA captioning models do not have. We further analyze where we outperform LiT in Section 4.2, and find that our method is particularly useful when there is little target data present in the pre-training data (see *count* in **Coverage Analysis**).

Runtime: Applying mLLMs using likelihoods (GPT-

	Kitti	EuroSAT	STL10	Country211	CIFAR 10	GTSRB	CIFAR 100	Caltech101	Resisc45	Flowers	Sun397	ImageNet 1k	Average	Pets	Cars	DTD	FGVC Aircraft	PatchCamelyon	Diabetic Retinopathy	MNIST	RenderedSST2	CLEVR Distance	CLEVR Count All	DMLAB	SVHN	Average
	High Count													Low Count												
Coverage	1.00	0.70	1.00	0.96	1.00	0.47	0.76	0.77	0.47	0.67	0.65	0.82	n/a	0.86	0.47	0.13	0.38	1.00	0.20	0.00	0.00	0.00	0.00	0.00	0.00	n/a
Count	2101.5k	24.5k	23.5k	22.2k	17.6k	17.3k	14.3k	7.8k	5.9k	3.9k	3.7k	2.5k	n/a	503	432	219	171.9	18	1.8	0.00	0.00	0.00	0.00	0.00	0.00	n/a
LiT	18.7	47.7	97.6	11.7	91.9	43.2	67.9	82.2	50.4	66.5	65.5	64.0	59.0	80.8	74.6	36.1	9.4	56.4	2.3	36.4	48.4	11.0	13.3	19.7	17.0	33.8
CLAMP(Ours)	24.2	47.8	96.4	12.7	88.4	31.8	65.3	81.1	42.6	64.9	66.4	63.0	57.0	78.3	78.7	49.3	16.3	50.2	4.4	45.7	51.5	11.2	14.9	14.9	28.3	37.0
ZS CLIP	15.2	45.0	97.0	15.9	91.8	43.5	67.6	84.9	53.4	65.0	69.2	64.7	59.4	86.6	82.3	47.6	17.0	52.5	2.3	33.9	53.7	24.5	21.5	14.7	19.0	38.0

Table 2. **Comparison based on concept count.** We sort datasets based on count of target classes within our LAION-200M subset. **Coverage** indicates the fraction of classes present within pre-training captions as a noun-phrase. **Count** counts the number of times the class set is present in the pre-training data captions, normalized by the number of classes. We then divide the datasets in half, into high count and low count datasets. We find that our method outperforms the LiT [55] baseline on the low count datasets by 3% on average and up to 13% (DTD). This shows the benefit of language pre-training for low-coverage datasets. For reference, we also add zero-shot CLIP-ViT-B 16, trained on LAION-400M (ZS CLIP). Even though the ZS CLIP model is trained on almost 1.5x the data, CLAMP approaches its performance. For more discussion, see Section 4.2.

Score) to do zero-shot classification is extremely inefficient; for example evaluating LLaVA on the ImageNet test set takes 12 hours 52 minutes on 10 GPUs. This is because each image needs to be forwarded through the LLM N times, where N is the number of classes. In contrast, it takes our method minutes for the same evaluation, as we can precompute the text embeddings ahead of time and we avoid the combinatorial explosion of early fusion.

Coverage Analysis One benefit of large scale pre-training is improved performance on data which is similar to the pre-training data, but less similar to the fine-tuning data. To check whether this is the case with our models, we utilize a metric from [53] called Concept Coverage to measure the presence of evaluation data in the pre-training dataset. The methodology uses spaCy¹. Following [53], we iterate over all the captions in our LAION subset and extract noun chunks, adding them into a dictionary. Finally, we filter out all noun chunks which appear fewer than 5 times. Then, we compute two metrics for each downstream dataset. First, we check how many class labels are present in the concept dictionary, and report this as fraction of all labels which we call **coverage**. Second, we compute the average number of pre-training images per class label in the target set, and call this **count**. We present the results in Table 2.

We can observe two things. First, there is a strong correlation between absolute performance and concept counts. When we split the evaluation datasets into “high count” and “low count”, “high count” has much higher average performance. Second, in the “low count” regime, our model outperforms the LiT baseline on average. For example, Cars only has about 432 samples per vehicle type, and our model outperforms the LiT baseline by over 4%. Similarly, our

pre-training data only has about 170 samples per aircraft type, and our model outperforms the LiT baseline by about 7%. In other words, the knowledge imported by the very large-scale text-based pre-training of an LLM allows us to generalize better than simply training on image-caption pairs. This highlights the benefits of LLM-based discriminative zero-shot classification.

Two datasets which appear to refute the rule are Kitti (with very high coverage and count, but lower performance) and PatchCamelyon (low coverage count, and also lower performance). PatchCamelyon is a dataset of histopathologic scans of lymph node sections annotated with a binary label indicating presence of cancer. This is a very difficult concept to describe in natural language; in fact humans trained to do this task must be highly specialized and undergo many years of training. Therefore, a model pre-trained on language data may not be appropriate. The second dataset is Kitti. Looking more closely at the Kitti class labels, we have the set {‘a photo of a photo i took of a car on my left or right side’, ‘a photo i took with a car nearby’, ‘a photo i took with a car in the distance.’, ‘a photo i took with no car.’}. These labels have nouns which are not discriminative, ‘car’ and ‘photo’, so the concept coverage heuristic is not fully relevant for this dataset. Instead the discriminative labels are adjectives corresponding to distance.

4.3. Generative Abilities of Finetuned models.

As one of our aims is to take steps towards model universality, an important metric is whether we retain generative capabilities in our models even after our discriminative fine-tuning. One could expect that contrastive adaptation would ruin the generative abilities of the model.

¹<https://spacy.io/>

Category	Prompt and Output
Creative writing	Prompt: Write a short story about a dragon who was evil and then saw the error in [sic] it's ways. Output: Once upon a time, in a land far, far away, there lived a dragon named Draco. Draco was known throughout the land as the most evil dragon that had ever lived. He would terrorize the countryside, burning down villages and towns, and eating anyone who crossed his path. The villagers lived in fear of Draco, and many had lost loved ones to his fiery breath. However, one day, Draco came across a group of villagers who were lost in the woods. They were terrified of him, but he saw that they were not armed and did not seem to be dangerous. He decided to let them go, and as they walked away, he heard a voice in his head telling him that he was wrong to terrorize the countryside. He realized that he had been evil all his life, and that he had no reason to be so cruel to others. From that day on, Draco changed his ways and became a kind and gentle dragon. He helped the villagers and protected them from other dangerous creatures in the woods. The villagers were amazed and grateful to Draco, and they all lived happily ever after. The end.
Factual Questions	Prompt: How was Anne Frank's diary discovered? Output: Anne Frank's diary was discovered by chance. In 1944, the Nazis occupied the Netherlands and the Frank family went into hiding in an attic above the office of the company where Anne's father worked. The attic was used as a storage room and was not used for several months. The employees of the company were moving out of the building and found the hidden attic and the hidden door to the attic. They found the diary and other documents hidden behind a false wall in the attic. They took the diary and other documents to the office manager, who recognized Anne Frank's name and realized the significance of what he had found. He kept the diary and other documents safe until after the war, when he returned to the office and found the diary among his papers. He then gave it to the Anne Frank Foundation, which published the diary and helped bring Anne Frank's story to the world.
Personal & professional development	Prompt: I sit in front of a computer all day. How do I manage and mitigate eye strain? Output: To manage and mitigate eye strain, follow these steps: 1. Take regular breaks: Every 25 minutes, take a 15-20 second break, look away from the screen, and focus on something 20 feet away. 3. Adjust your monitor: Position your monitor at a comfortable distance from your eyes, and adjust the brightness and contrast to reduce glare. 4. Use proper lighting: Ensure adequate lighting in your workspace to reduce glare and reflections. 5. Blink frequently: Blinking helps to keep your eyes moist and reduces the strain. 6. Consider computer glasses: If you already wear prescription glasses, consider computer glasses with a higher add power for reading and computer work. 7. Practice the 20-20-20 rule: Every 20 minutes, look at something 20 feet away for 20 seconds. 8. Consider blue light filter glasses: Blue light filter glasses can help reduce digital eye strain. 9. Maintain good posture: Sit up straight, with your feet flat on the floor, and your shoulders relaxed. 10. Exercise regularly and maintain a healthy diet: A healthy lifestyle can help reduce eye strain and other health issues.
Casual advice & recommendations	Prompt: I keep losing my keys. How can I keep track of them? Output: keys, you can try using a key finder or a key tracker. These devices attach to your keys and allow you to locate them using your smartphone or other devices. Some popular options include Tile, Chip, and KeySmart.
Reasoning (math/ problem-solving)	Prompt: User: A jar contains 60 jelly beans, If 35% of the jelly beans are removed how many are left in the jar? Assistant: If 35% of the jelly beans are removed, then the number of jelly beans left in the jar is $60 - (35\% \text{ of } 60) = 60 - 21 = 39$. User: can you expand your answer to show your reasoning? Output: Sure! To find the number of jelly beans left in the jar after 35% of them have been removed, we need to first calculate what 35% of 60 is. We can do this by multiplying 60 by 35% (which we can express as a fraction): $60 \times 35\% = 60 \times 35/100 = 21$. Then, we subtract 21 from 60 to find the number of jelly beans left: $60 - 21 = 39$. So there would be 39 jelly beans left in the jar after 35% of them have been removed.

Table 3. Qualitative Examples of our finetuned LLM on NLP Generation

We show qualitative examples in Table 3. From inspection, the results are quite reasonable. We also measure performance on MMLU[15] (an LLM test of 57 tasks including elementary mathematics, US history, computer science, law, and more. To attain high accuracy on this test, models must possess extensive world knowledge and problem solving ability), DROP[11] (a math question answering benchmark), and BBH[45] (which includes 23 exceptionally hard tasks). Quantitatively, Vicuna performance on MMLU drops from 55.74 before fine-tuning to 34.91, which is better than even much larger LLMs from a previous generation. For example, OPT-66B [57] achieves an MMLU of 26.99. On DROP the degradation is lesser, from 27.73 to 24.93. On BBH the metric falls from 38.42 to 32.11. We emphasize that if the user wants to maximize generative performance, they can disable the LoRA parameters during inference.

4.4. Model Analysis

Effect of Data Scale: Data scale is an extremely important part of vision-language pretraining in general. We confirm that data scale remains very important even with our strong language prior by subsampling our data and training. In Figure 3, we show top-1 ImageNet performance as we scale data. As data grows, so does top-1 accuracy.

Regularized Fine-tuning Alternatives: One of our aims is to retain the generative ability of the large language model; and as a result we regularize our fine-tuning so that our model does not catastrophically forget that capability. We choose to combine Read-Only Prompts (Section 3.1.1) and LoRA (Section 3.1.3). However, this is not the only choice. In fact, Qi et al. [37] show that combining LayerNorm tuning with prompt tuning is amongst the most effective tuning methods. LayerNorm tuning is even lighter weight than tun-

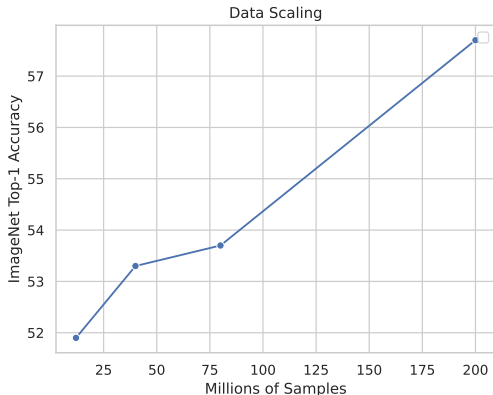


Figure 3. **Scaling training data.** We confirm that data scale remains very important even with our strong language prior by subsampling our data and training. As data grows, so does top-1 zero-shot ImageNet accuracy.

	ImageNet 1k	Cars	CIFAR 100	DTD	EuroSAT	Flowers	SVHN	Average
LoRA	63.0	78.7	65.3	49.3	47.8	64.9	28.3	56.8
LN-Prefix	61.1	74.8	68.1	52.7	42.8	59.4	31.6	55.8

Table 4. **LN-Prefix Comparison.** We compare with an alternative to LoRA, which is prefix tuning combined with LayerNorm parameter tuning. We find that LoRA outperforms the LN-Prefix combination.

	Distillation	ImageNet21k	LoRA	ImageNet 1k	Average
All	✓	✓	✓	60.4	47.2
- LoRA	✓	✓		57.9	47.0
- Distillation		✓	✓	57.7	45.7
- ImageNet21k	✓		✓	60.4	46.5

Table 5. **Ablation Study.** We observe the difference in performance when we remove distillation, ImageNet-21k, and LoRA. We compute both average performance across 24 datasets, and ImageNet1k performance. Each component is important.

ing LoRA parameters, and therefore if effective would be well-suited for our purposes. However, as we show in Table 4, in our setting LoRA is a stronger adaptation method than prompt tuning combined with LayerNorm tuning. In our experiments, for the Prompt + LayerNorm tuning experiments, we add 12 soft prompt tokens to the beginning of the model, and un-freeze all LayerNorm parameters.

Ablations: In Table 5, we ablate several components of CLAMP; Distillation, LoRA, and ImageNet21k. We find that each piece is important for the average performance across all 24 evaluation datasets. Interestingly, we find that adding ImageNet-21k does not improve ImageNet-1k performance, likely because there is no overlap in class labels.

5. Conclusion

In this work, we explore how to make multi-modal LLMs work more effectively for zero-shot classification. We start by showing that SOTA models such as LLaVA or miniGPT-4 do not perform well for zero-shot classification, and underperform models trained with a contrastive captioning loss by over 10%. We then show that we can add discriminative abilities to pre-trained Large Language Models by aligning an LLM with a pre-trained vision encoder with a contrastive loss, and only performing a lightweight finetuning. We show that our contrastively trained LLM is effective at zero-shot classification, and outperforms contrastively trained baselines in the regime where pre-training data does not overlap with the evaluation dataset. Finally, we show that, despite fine-tuning, our model retains generative capabilities.

References

- [1] Jean-Baptiste Alayrac, Jeff Donahue, Pauline Luc, Antoine Miech, Iain Barr, Yana Hasson, Karel Lenc, Arthur Mensch, Katherine Millican, Malcolm Reynolds, et al. Flamingo: a visual language model for few-shot learning. *Advances in Neural Information Processing Systems*, 35:23716–23736, 2022. 2, 5
- [2] Soravit Changpinyo, Piyush Sharma, Nan Ding, and Radu Soricut. Conceptual 12M: Pushing web-scale image-text pre-training to recognize long-tail visual concepts. In *CVPR*, 2021. 5, 12
- [3] Ting Chen, Simon Kornblith, Mohammad Norouzi, and Geoffrey Hinton. A simple framework for contrastive learning of visual representations. In *International conference on machine learning*, pages 1597–1607. PMLR, 2020. 2
- [4] Gong Cheng, Junwei Han, and Xiaoqiang Lu. Remote sensing image scene classification: Benchmark and state of the art. *Proceedings of the IEEE*, 105(10):1865–1883, 2017. 13
- [5] Wei-Lin Chiang, Zhuohan Li, Zi Lin, Ying Sheng, Zhanghao Wu, Hao Zhang, Lianmin Zheng, Siyuan Zhuang, Yonghao Zhuang, Joseph E. Gonzalez, Ion Stoica, and Eric P. Xing. Vicuna: An open-source chatbot impressing gpt-4 with 90%* chatgpt quality, 2023. 3, 5
- [6] Hyung Won Chung, Le Hou, Shayne Longpre, Barret Zoph, Yi Tay, William Fedus, Yunxuan Li, Xuezhi Wang, Mostafa Dehghani, Siddhartha Brahma, et al. Scaling instruction-finetuned language models. *arXiv preprint arXiv:2210.11416*, 2022. 2
- [7] M. Cimpoi, S. Maji, I. Kokkinos, S. Mohamed, and A. Vedaldi. Describing textures in the wild. In *Proceedings of the IEEE Conf. on Computer Vision and Pattern Recognition (CVPR)*, 2014. 12
- [8] Adam Coates, Andrew Ng, and Honglak Lee. An Analysis of Single Layer Networks in Unsupervised Feature Learning. In *AISTATS*, 2011. https://cs.stanford.edu/~acoates/papers/coatesleeng_aistats_2011.pdf. 12
- [9] Wenliang Dai, Junnan Li, Dongxu Li, Anthony Meng Huat Tiong, Junqi Zhao, Weisheng Wang, Boyang Li, Pascale Fung, and Steven Hoi. Instructblip: Towards general-purpose vision-language models with instruction tuning, 2023. 1, 2
- [10] Jia Deng, Wei Dong, Richard Socher, Li-Jia Li, Kai Li, and Li Fei-Fei. Imagenet: A large-scale hierarchical image database. In *2009 IEEE conference on computer vision and pattern recognition*, pages 248–255. Ieee, 2009. 4, 5, 12
- [11] Dheeru Dua, Yizhong Wang, Pradeep Dasigi, Gabriel Stanovsky, Sameer Singh, and Matt Gardner. DROP: A reading comprehension benchmark requiring discrete reasoning over paragraphs. In *Proc. of NAACL*, 2019. 8
- [12] Li Fei-Fei, Rob Fergus, and Pietro Perona. Learning generative visual models from few training examples: An incremental bayesian approach tested on 101 object categories. *Computer Vision and Pattern Recognition Workshop*, 2004. 12
- [13] Andreas Geiger, Philip Lenz, and Raquel Urtasun. Are we ready for autonomous driving? the kitti vision benchmark suite. In *Conference on Computer Vision and Pattern Recognition (CVPR)*, 2012. 4, 13
- [14] Patrick Helber, Benjamin Bischke, Andreas Dengel, and Damian Borth. Eurosat: A novel dataset and deep learning benchmark for land use and land cover classification, 2017. 4, 12
- [15] Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. Measuring massive multitask language understanding. *arXiv preprint arXiv:2009.03300*, 2020. 4, 8
- [16] Neil Houlsby, Andrei Giurgiu, Stanislaw Jastrzebski, Bruna Morrone, Quentin De Laroussilhe, Andrea Gesmundo, Mona Attariyan, and Sylvain Gelly. Parameter-efficient transfer learning for nlp. In *International Conference on Machine Learning*, pages 2790–2799. PMLR, 2019. 2
- [17] Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. Lora: Low-rank adaptation of large language models. *arXiv preprint arXiv:2106.09685*, 2021. 2
- [18] Gabriel Ilharco, Mitchell Wortsman, Ross Wightman, Cade Gordon, Nicholas Carlini, Rohan Taori, Achal Dave, Vaishaal Shankar, Hongseok Namkoong, John Miller, Hananeh Hajishirzi, Ali Farhadi, and Ludwig Schmidt. Openclip, 2021. If you use this software, please cite it as below. 13
- [19] Justin Johnson, Bharath Hariharan, Laurens van der Maaten, Li Fei-Fei, C Lawrence Zitnick, and Ross Girshick. CLEVR: A diagnostic dataset for compositional language and elementary visual reasoning. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2017. 12
- [20] Kaggle and EyePacs. Kaggle diabetic retinopathy detection, 2015. 12
- [21] Prannay Khosla, Piotr Teterwak, Chen Wang, Aaron Sarna, Yonglong Tian, Phillip Isola, Aaron Maschiot, Ce Liu, and Dilip Krishnan. Supervised contrastive learning. *Advances in neural information processing systems*, 33:18661–18673, 2020. 2
- [22] Jonathan Krause, Michael Stark, Jia Deng, and Li Fei-Fei. 3d object representations for fine-grained categorization. In *4th International IEEE Workshop on 3D Representation and Recognition (3dRR-13)*, Sydney, Australia, 2013. 1, 4, 12
- [23] Alex Krizhevsky. Learning multiple layers of features from tiny images. Technical report, 2009. 12
- [24] Yann LeCun, Corinna Cortes, and CJ Burges. Mnist handwritten digit database. *ATT Labs [Online]*. Available: <http://yann.lecun.com/exdb/mnist>, 2, 2010. 12
- [25] Dongjun Lee, Seokwon Song, Jihee Suh, Joonmyeong Choi, Sanghyeok Lee, and Hyunwoo J. Kim. Read-only prompt optimization for vision-language few-shot learning. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2023. 2, 3
- [26] Brian Lester, Rami Al-Rfou, and Noah Constant. The power of scale for parameter-efficient prompt tuning. *arXiv preprint arXiv:2104.08691*, 2021. 2
- [27] Zhiqiu Lin, Xinyue Chen, Deepak Pathak, Pengchuan Zhang, and Deva Ramanan. Visualgptscore: Visio-linguistic reasoning with multimodal generative pre-training scores. *arXiv preprint arXiv:2306.01879*, 2023. 2, 5

- [28] Haotian Liu, Chunyuan Li, Yuheng Li, and Yong Jae Lee. Improved baselines with visual instruction tuning, 2023. [5](#)
- [29] Haotian Liu, Chunyuan Li, Qingyang Wu, and Yong Jae Lee. Visual instruction tuning. *arXiv preprint arXiv:2304.08485*, 2023. [1](#), [2](#), [4](#), [5](#)
- [30] Xiao Liu, Kaixuan Ji, Yicheng Fu, Weng Lam Tam, Zhengxiao Du, Zhilin Yang, and Jie Tang. P-tuning v2: Prompt tuning can be comparable to fine-tuning universally across scales and tasks. *arXiv preprint arXiv:2110.07602*, 2021. [2](#)
- [31] Subhansu Maji, Esa Rahtu, Juho Kannala, Matthew Blaschko, and Andrea Vedaldi. Fine-grained visual classification of aircraft. *arXiv preprint arXiv:1306.5151*, 2013. [4](#), [12](#)
- [32] Yuval Netzer, Tao Wang, Adam Coates, Alessandro Bis-sacco, Bo Wu, and Andrew Y Ng. Reading digits in natural images with unsupervised feature learning. 2011. [13](#)
- [33] M-E. Nilsback and A. Zisserman. Automated flower classification over a large number of classes. In *Proceedings of the Indian Conference on Computer Vision, Graphics and Image Processing*, 2008. [12](#)
- [34] Aaron van den Oord, Yazhe Li, and Oriol Vinyals. Representation learning with contrastive predictive coding. *arXiv preprint arXiv:1807.03748*, 2018. [2](#)
- [35] R OpenAI. Gpt-4 technical report. *arXiv*, pages 2303–08774, 2023. [1](#), [2](#)
- [36] O. M. Parkhi, A. Vedaldi, A. Zisserman, and C. V. Jawahar. Cats and dogs. In *IEEE Conference on Computer Vision and Pattern Recognition*, 2012. [13](#)
- [37] Wang Qi, Yu-Ping Ruan, Yuan Zuo, and Taihao Li. Parameter-efficient tuning on layer normalization for pre-trained language models. *arXiv preprint arXiv:2211.08682*, 2022. [8](#)
- [38] Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, et al. Learning transferable visual models from natural language supervision. In *International conference on machine learning*, pages 8748–8763. PMLR, 2021. [1](#), [2](#), [3](#), [4](#), [5](#), [12](#)
- [39] Christoph Schuhmann, Richard Vencu, Romain Beaumont, Robert Kaczmarczyk, Clayton Mullis, Aarush Katta, Theo Coombes, Jenia Jitsev, and Aran Komatsuzaki. Laion-400m: Open dataset of clip-filtered 400 million image-text pairs. *arXiv preprint arXiv:2111.02114*, 2021. [5](#), [12](#)
- [40] Piyush Sharma, Nan Ding, Sebastian Goodman, and Radu Soricut. Conceptual captions: A cleaned, hypernymed, image alt-text dataset for automatic image captioning. In *Proceedings of ACL*, 2018. [5](#), [12](#)
- [41] Karan Singhal, Shekoofeh Azizi, Tao Tu, S Sara Mahdavi, Jason Wei, Hyung Won Chung, Nathan Scales, Ajay Tanwani, Heather Cole-Lewis, Stephen Pfohl, et al. Large language models encode clinical knowledge. *Nature*, 620 (7972):172–180, 2023. [2](#)
- [42] Richard Socher, Alex Perelygin, Jean Wu, Jason Chuang, Christopher D. Manning, Andrew Ng, and Christopher Potts. Recursive deep models for semantic compositionality over a sentiment treebank. In *Proceedings of the 2013 Conference on Empirical Methods in Natural Language Processing*, pages 1631–1642, Seattle, Washington, USA, 2013. Association for Computational Linguistics. [12](#)
- [43] Johannes Stallkamp, Marc Schlipsing, Jan Salmen, and Christian Igel. Man vs. computer: Benchmarking machine learning algorithms for traffic sign recognition. *Neural networks*, 32:323–332, 2012. [12](#)
- [44] Ximeng Sun, Pengchuan Zhang, Peizhao Zhang, Hardik Shah, Kate Saenko, and Xide Xia. Dime-fm : Distilling multimodal and efficient foundation models. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 15521–15533, 2023. [3](#), [4](#)
- [45] Mirac Suzgun, Nathan Scales, Nathanael Schärli, Sebastian Gehrmann, Yi Tay, Hyung Won Chung, Aakanksha Chowdhery, Quoc V Le, Ed H Chi, Denny Zhou, , and Jason Wei. Challenging big-bench tasks and whether chain-of-thought can solve them. *arXiv preprint arXiv:2210.09261*, 2022. [8](#)
- [46] Bart Thomee, David A Shamma, Gerald Friedland, Benjamin Elizalde, Karl Ni, Douglas Poland, Damian Borth, and Li-Jia Li. Yfcc100m: The new data in multimedia research. *Communications of the ACM*, 59(2):64–73, 2016. [12](#)
- [47] Yonglong Tian, Dilip Krishnan, and Phillip Isola. Contrastive multiview coding. In *Computer Vision—ECCV 2020: 16th European Conference, Glasgow, UK, August 23–28, 2020, Proceedings, Part XI 16*, pages 776–794. Springer, 2020. [2](#)
- [48] Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, et al. Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288*, 2023. [1](#), [2](#), [3](#)
- [49] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in neural information processing systems*, 30, 2017. [2](#)
- [50] Shijie Wu, Ozan Irsoy, Steven Lu, Vadim Dabravolski, Mark Dredze, Sebastian Gehrmann, Prabhajan Kambadur, David Rosenberg, and Gideon Mann. Bloomberggpt: A large language model for finance. *arXiv preprint arXiv:2303.17564*, 2023. [2](#)
- [51] Guangxuan Xiao, Yuandong Tian, Beidi Chen, Song Han, and Mike Lewis. Efficient streaming language models with attention sinks. *arXiv preprint arXiv:2309.17453*, 2023. [4](#)
- [52] J. Xiao, J. Hays, K. A. Ehinger, A. Oliva, and A. Torralba. Sun database: Large-scale scene recognition from abbey to zoo. In *2010 IEEE Computer Society Conference on Computer Vision and Pattern Recognition*, pages 3485–3492, 2010. [12](#)
- [53] Jianwei Yang, Chunyuan Li, Pengchuan Zhang, Bin Xiao, Ce Liu, Lu Yuan, and Jianfeng Gao. Unified contrastive learning in image-text-label space. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 19163–19173, 2022. [6](#), [13](#)
- [54] Xiaohua Zhai, Joan Puigcerver, Alexander Kolesnikov, Pierre Ruysen, Carlos Riquelme, Mario Lucic, Josip Djolonga, Andre Susano Pinto, Maxim Neumann, Alexey Dosovitskiy, Lucas Beyer, Olivier Bachem, Michael Tschannen, Marcin Michalski, Olivier Bousquet, Sylvain Gelly, and Neil Houlsby. The visual task adaptation benchmark. 2019. [12](#)

- [55] Xiaohua Zhai, Xiao Wang, Basil Mustafa, Andreas Steiner, Daniel Keysers, Alexander Kolesnikov, and Lucas Beyer. Lit: Zero-shot transfer with locked-image text tuning. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 18123–18133, 2022. [2](#), [4](#), [5](#), [6](#)
- [56] Xiaohua Zhai, Basil Mustafa, Alexander Kolesnikov, and Lucas Beyer. Sigmoid loss for language image pre-training. *arXiv preprint arXiv:2303.15343*, 2023. [2](#)
- [57] Susan Zhang, Stephen Roller, Naman Goyal, Mikel Artetxe, Moya Chen, Shuohui Chen, Christopher Dewan, Mona Diab, Xian Li, Xi Victoria Lin, Todor Mihaylov, Myle Ott, Sam Shleifer, Kurt Shuster, Daniel Simig, Punit Singh Koura, Anjali Sridhar, Tianlu Wang, and Luke Zettlemoyer. Opt: Open pre-trained transformer language models, 2022. [8](#)
- [58] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric. P Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. Judging llm-as-a-judge with mt-bench and chatbot arena, 2023. [2](#)
- [59] Deyao Zhu, Jun Chen, Xiaoqian Shen, Xiang Li, and Mohamed Elhoseiny. Minigpt-4: Enhancing vision-language understanding with advanced large language models. *arXiv preprint arXiv:2304.10592*, 2023. [1](#), [2](#), [3](#), [4](#), [5](#)

A. Dataset Details

We detail the datasets used in our paper, both for training and evaluation.

A.1. Training data

We use the following image-text datasets for pre-training:

Conceptual Captions 3M [40]: A dataset of 3.3 million images, with the text captions being HTML alt-text. Both images and text are filtered to retain high quality images.

Conceptual Captions 12M [2]: A larger-scale version of Conceptual Captions 3M, containing 10 million images. The image and text filter used for collecting Conceptual Captions 3 million are made less strict, allowing for a noisier yet larger dataset.

LAION-200M[39]: We subsample the LAION-400M dataset uniformly and at random, collecting a total of 264662608 samples, due to downloading failures and dead links. The LAION-400M scrapes the Common Crawl dataset, and filters out samples which are too dissimilar according to a cosine-similarity computed using an OpenAI CLIP model.

ImageNet-21K [10] ImageNet-22k is the full ImageNet-21k dataset, with ImageNet-1k classes removed to fairly evaluate the zero-shot capabilities of trained models on the ImageNet-1k dataset.

A.2. Evaluation Datasets

Below we describe the zero-shot evaluation dataset we use for evaluation.

ImageNet 1k[10]: ImageNet1k is a natural image dataset consisting of 1000 object classes, evenly distributed over 50,000 images.

Stanford Cars[22]: The Stanford cars dataset contains 8041 test images, over 196 classes of cars. Each class is a Make, Model, Year combination, for example “Hyundai Veloster Hatchback 2012”. Some distinctions are quite fine grained; for example “Aston Martin V8 Vantage Convertible 2012” versus “Aston Martin V8 Vantage Coupe 2012”.

Country211 [38]: Country211 is a subset of YFCC100M [46] to find geotagged images from 211 countries. The dataset is balanced across classes, with 100 samples from each class.

FGVC Aircraft [31]: The aircraft dataset contains 3333 test images divided across 100 aircraft models *e.g.* 777-300 or DC-10. It’s very fine-grained, the differences between different generations of the same aircraft (747-300 vs 747-400) are quite subtle.

GTSRB [43]: GTSRB is a classification benchmark of German Traffic Signs into classes like “red and white circle 20 kph speed limit”. There are 12630 evaluation images and 43 classes.

MNIST [24]: MNIST is a dataset of black-and-white hand-written digits, with 10 classes split across 10000 test images.

RenderedSST2 [38] : RenderedSST2 contains text rendered in images, from the Stanford Sentiment Treebank v2 dataset[42]. The sentences are pulled from movie reviews, and the task is a binary classification into positive and negative sentiment movies. There are 1821 samples in the test set.

STL10 [8]: STL10 is a natural image dataset of 96x96 images in 10 classes. There 800 images per-class for a total of 8000 test images.

SUN397 [52]: SUN397 is a natural image dataset of scenes, instead of objects. For example, labels include “airport terminal” and “jewelry shop”. The evaluation set is balanced, and contains 50 samples per class for a total of 19850 images.

Caltech101 [12]: Caltech101 is a natural image dataset of 101 classes and a background class, each labelled with a single object. There are 6085 test samples.

CIFAR-10 [23]: CIFAR-10 is a natural image dataset of 32x32 images, with each sample belonging to one of ten classes. There are 10000 samples.

CIFAR-100 [23]: Similar to CIFAR-100 but with more classes, CIFAR-100 is a natural image dataset of 32x32 images, with each sample belonging to one of one hundred classes. There are 10000 samples.

CLEVR Distance [19]: A synthetic dataset of objects, where the task is a 5 way classification of distances labels “{‘very nearby’, ‘nearby’, ‘near’, ‘distant’, ‘very distant’}”, which correspond to the distance of the closest object. There are 15000 test samples.

CLEVR Count [19]: A synthetic dataset of objects, where the task is an 8 way classification of labels, corresponding to object counting. There are 15000 test samples, which are the same as for the CLEVR distance classification task.

Diabetic Retinopathy [20]: This dataset is a medical imaging dataset of retinas, with a 5 way classification problem corresponding to the severity of the diabetic retinopathy present in the image. There are 42670 test images.

DMLAB [54]: DMLAB consists of synthetic images, and there are 6 classes, consisting of positive rewards(apples and melons) or negative rewards(lemons), each of which can be nearby, far, or very far. There are 22735 test images.

Describable Textures [7]: Describable Textures is 1880 test images of textures, divided into 47 classes. Class labels are labelled with texture descriptions such as “wrinkled”, “marbled”, or “braided”.

EuroSAT [14]: EuroSAT contains images of satellite imagery, split into 10 classes of land use like “forest” or “pasture land”. The test set contains 5400 samples.

Oxford Flowers [33]: Oxford Flowers is a dataset of flower species; with 102 classes and 6149 samples.

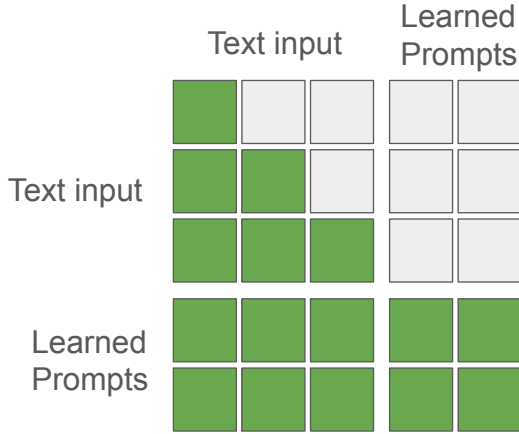


Figure 4. **Read only prompts.** The attention we use. The Learned Prompts can attend to all positions in the sequence, while text tokens can only attend to tokens in positions before.

KITTI [13]: KITTI consists of test images divided into four classes {‘a photo i took of a car on my left or right side.’, ‘a photo i took with a car nearby.’, ‘a photo i took with a car in the distance.’, ‘a photo i took with no car.’}, corresponding to the distance of the closest car in photos taken from the perspective of a car driver.

PatchCamelyon [?]: PatchCamelyon is a challenging dataset of microscope slide images, split evenly between cancerous and non-cancerous samples. There are 32768 test images.

Oxford-IIT Pets [36]: 3669 test images of cats and dogs, split into 37 classes pertaining to different breeds.

Resisc45[4]: A satellite imagery dataset, with 45 classes which are mixture of natural (*e.g.* ‘mountain’) and man-made (‘harbor’) classes. There are a total of 6300 evaluation images.

SVHN[32]: This is a dataset testing OCR capabilities. There are ten classes, corresponding to ten digits. The images are 32x32 and are cropposed house numbers. There are 26,032 test images.

B. Read Only Prompts Attention Mask

In this section, we illustrate the Attention Mask of the Read-Only prompts. As we discuss in the main paper, causal attention masks mean that tokens appended to the end of an input sequence can only attend to tokens coming before them. In the case of read-only prompts, this is desirable in order to not shift the representations of the text tokens while learning the soft tokens. We additionally modify the attention mask as illustrated in Figure 4, such that read-only prompts can attend to each other as well.

```

1 import pandas as pd
2 import spacy
3 from tqdm import tqdm
4 import pickle
5
6 nlp = spacy.load("en_core_web_sm", disable=['ner',
7         , 'lemmatizer', 'textcat'])
8 dataset = pd.read_csv('cc12m.tsv', delimiter='\t')
9
10 concept_dict = {}
11 for i, txt in tqdm(enumerate(nlp.pipe(iter(
12     dataset['caption'])))):
13     for n in txt.noun_chunks:
14         chunk = str(n).lower().strip()
15         concept_dict[chunk] = concept_dict.get(
16             chunk, 0) + 1
17
18 with open('concept_dict.pkl', 'wb') as handle:
19     pickle.dump(concept_dict, handle)

```

Figure 5. **Generating Pre-training Concept Dictionaries.** We iterate through captions in the pre-training data, use spaCy to extract noun chunks, and populate a dictionary.

C. Training Details

We build on top of OpenCLIP[18], and provide hyperparameters in Table 6.

D. Coverage Analysis

D.1. Pseudocode

We generate concept count dictionaries by counting the frequency of noun-chunks in the pre-training data, following UniCL [53]. Then, we extract noun-chunks from class labels in target datasets and count the frequency of noun chunks present in the pretraining data. We only consider a label present in the pre-training data if a noun-chunk is present more than 5 times. We then compute two metrics: what percentage of target labels are present in the pre-training (**coverage**) and how many times the noun chunks present in the target class label list are present in the pre-training data. We show pseudo-code for constructing conception dictionary from pre-training data (Figure 5) and computing **count** and **coverage** (Figure 6).

D.2. Statistics for other pre-training datasets

In the main paper, we compute coverage and count statistics for LAION-200M, but we also pretrain on ImageNet-21k, CC3M, and CCM12M. Therefore, we provide statistics below. As one can see, overall the statistics are heavily correlated for all the datasets.

¹<https://huggingface.co/lmsys/vicuna-13b-v1.5>

Parameter	Value
Text Encoder Arch.	CLIP ViT-B-16
Text Encoder Init.	Random
Vision Encoder Arch.	ViT-B-16
Vision Encoder Init.	openclip, laion400m
Batch Size	8192
Optimizer	AdamW
Learning Rate	0.0005
Weight Decay	0.5
Training Steps	59317
Warmup Steps	1220
Learning Rate Schedule	cosine
Num Learnable Prompts	n/a
LoRA Rank	n/a
LoRA dropout	n/a
LoRA alpha	n/a
Gradient Clipping Norm	1

(a) LiT Training Hyperparameters

Parameter	Value
Text Encoder Arch.	Vicuna 13B
Text Encoder Init.	lmsys-1.5 ¹
Vision Encoder Arch.	ViT-B-16
Vision Encoder Init.	openclip, laion400m
Batch Size	8192
Optimizer	AdamW
Learning Rate	0.0005
Weight Decay	0.5
Training Steps	59317
Warmup Steps	1220
Learning Rate Schedule	cosine
Num Learnable Prompts	24
LoRA Rank	16
LoRA dropout	0.1
LoRA alpha	16
Gradient Clipping Norm	1

(b) CLAMP training hyperparameters

Table 6. Training hyperparameters.

Pretraining Data	Metric	Kitti	EuroSAT	STL10	Country211	CIFAR 10	GTSRB	CIFAR 100	Caltech101	Resisc45	Flowers	Sun397	ImageNet 1k	Pets	Cars	DTD	FGVC Aircraft	PatchCamelyon	Diabetic Retinopathy	MNIST	RenderedSST2	CLEVR Distance	CLEVR Count	DMLAB	SVHN
LAION-200M	Coverage	1.00	0.70	1.00	0.96	1.00	0.47	0.76	0.77	0.47	0.67	0.65	0.82	0.86	0.47	0.13	0.38	1.00	0.20	0.00	0.00	0.00	0.00	0.00	0.00
	Count	2101.5k	24.5k	23.5k	22.2k	17.6k	17.3k	14.3k	7.8k	5.9k	3.9k	3.7k	2.5k	503	432	219	172	18	2	0	0	0	0	0	0
CC3M	Coverage	1.00	0.60	1.00	0.06	1.00	0.44	0.72	0.58	0.47	0.16	0.44	0.41	0.11	0.05	0.04	0.06	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
	Count	83.6k	2.8k	2.2k	121	1.9k	21.9k	2.1k	416	730	95	190	158	47	2	2	42	0	0	0	0	0	0	0	0
CC12M	Coverage	1.00	0.70	1.00	0.86	1.00	0.44	0.76	0.72	0.47	0.38	0.58	0.70	0.73	0.24	0.06	0.19	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
	Count	373.0k	3.6k	5.1k	2.4k	4.5k	224.2k	2.8k	1.2k	1.0k	486	671	361	120	134	15	152	0	0	0	0	0	0	0	0
IM-21k	Coverage	1.00	0.10	1.00	0.07	1.00	0.14	0.62	0.45	0.27	0.34	0.29	0.13	0.05	0.03	0.04	0.01	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
	Count	650	65	1.5k	74	1.3k	423	715	452	217	300	247	130	35	27	28	7	0	0	0	0	0	0	0	0

Table 7. **Pretraining Coverage and Count Statistics.** We compute coverage and count statistics for our other pre-training datasets, in addition to LAION-200M. We find that these statistics follow similar trends in LAION-200M, CC3M, CC12M, and ImageNet-21k.

```

1
2 import spacy
3 from tqdm import tqdm
4 import pickle
5
6 nlp = spacy.load("en_core_web_sm", disable=['ner',
7     , 'lemmatizer', 'textcat'])
8
9 def read_file(class_list, dict_loc):
10
11     with open(dict_loc, 'rb') as f:
12         concept_dict = pickle.load(f)
13
14
15     coverage = 0
16     count = 0
17     num_classes = 0
18     try:
19         with open(class_list, 'r') as file:
20             for line in file:
21                 l = line.strip().lower()
22                 noun_chunks = nlp(l).noun_chunks
23                 local_count = 0
24                 present = 0
25                 for c in noun_chunks:
26                     count = concept_dict[c]
27                     if chunk_count > 5:
28                         local_count += count
29                         present = 1
30                     else:
31                         local_count += 0
32
33                 coverage += present
34                 count += local_count
35                 num_classes += 1
36
37             print('Coverage: {}'.format(coverage/
38 num_classes))
39             print('Count: {}'.format(count/
40 num_classes))

```

Figure 6. Generating Concept Count and Coverage. Pseudo-code for computing coverage and count statistics for a target dataset. We iterate through target classnames, extract noun chunks using spaCy, and check for their presence in the dictionary computed in Figure 5.